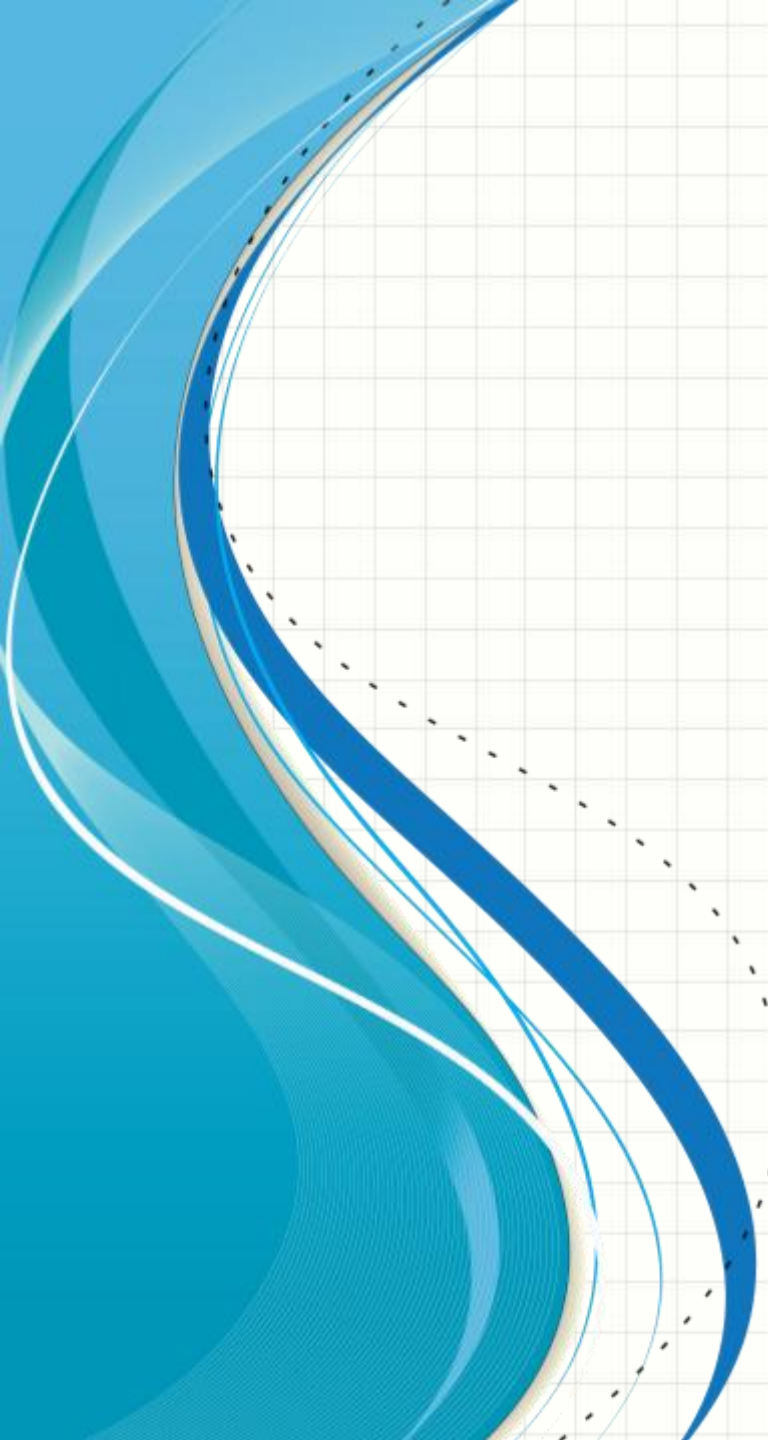




CHAPTER # 1

DIGITAL SYSTEMS

AND

BINARY NUMBERS

Lecturer: Noman Al Hassan
Email: noman44m@hotmail.com

Objective of Lecture

After studying this chapter, you should be able to know

- Analog and Digital Systems.
- Logic Levels.
 - TTL.
 - CMOS.
- Numbering Systems (Conversion examples).
 - Decimal System.
 - Binary System.
 - Octal System.
 - Hexadecimal System.
- Number Base Conversions.
- Comparison.
- Applications.

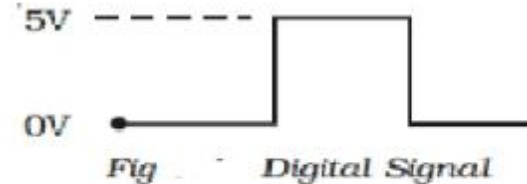
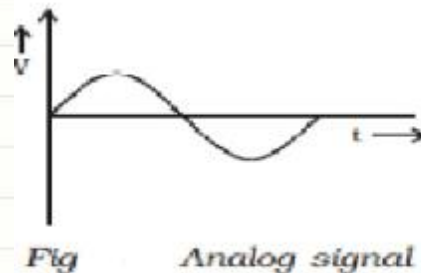
Objective of Lecture

After studying this chapter, you should be able to know

- Arithmetic Operations.
 - Binary.
 - Octal.
 - Hex.
- Complements.(Radix complement and Diminished radix complement)
 - 1's Complements.
 - 2's Complements.
 - 9's Complements.
 - 10's Complements.
- Signed and Unsigned binary numbers.
- Binary Coded Decimal.
- Error Detection.
 - Gray Code.
 - ASCII Code.

Analog & Digital Systems

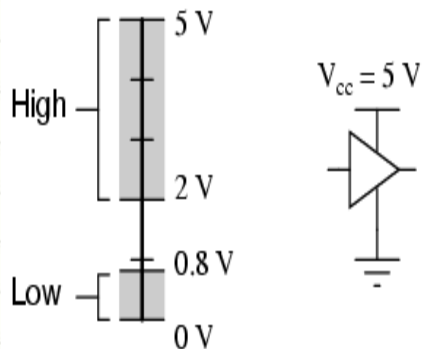
- **Analogue systems** with a continuously variable signal.
- **Digital systems** represent information using a binary **system**, where data can assume one of only two possible values: zero or one.



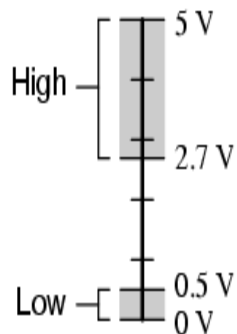
Logic Levels

- A **logic level** is one of several states that a digital signal can possess. Usually, the term refers to binary **logic** in which two **levels** exist:
 - **logic 1** (also called the high state)
 - **logic 0** (also called the low state).

Acceptable TTL gate input signal levels

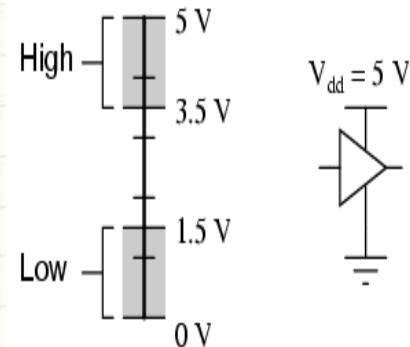


Acceptable TTL gate output signal levels



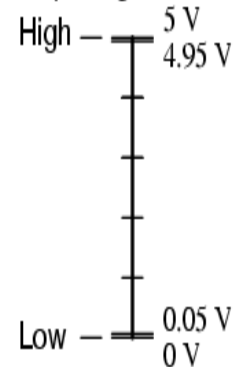
Transistor-transistor logic
(TTL)

Acceptable CMOS gate input signal levels



Complementary metal oxide semiconductor
(CMOS)

Acceptable CMOS gate output signal levels



Numbering Systems

Numbering Systems		
System	Base	Digits
Binary	2	0 1
Octal	8	0 1 2 3 4 5 6 7
Decimal	10	0 1 2 3 4 5 6 7 8 9
Hexadecimal	16	0 1 2 3 4 5 6 7 8 9 A B C D E F

Decimal Number System

- Base = 10
 - 10 digits { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }
- Digit Position
 - Integer
- Digit Weight
 - Weight = $(Base)^{Position}$
- Magnitude
 - Sum of “Digit x Weight”
- Formal Notation

2	1	0		-1	-2
5	1	2	.	7	4
100	10	1		0.1	0.01
			.		
500	10	2		0.7	0.04

$$d_2 * B^2 + d_1 * B^1 + d_0 * B^0 + d_{-1} * B^{-1} + d_{-2} * B^{-2}$$

$$(512.74)_{10}$$

Binary Number System

- In our Daily life we use decimal system but computer can only understand binary numbering system.
- Computer represent all kind of data and information in binary numbers.
- Base = 2
 - 2 digits { 0, 1 }, called *binary digits* or "*bits*"

- Weights

- Weight = $(Base)^{Position}$

- Magnitude

- Sum of "*Bit x Weight*"

- Formal Notation

- Groups of bits

4 bits = *Nibble*

1 0 1 1

8 bits = *Byte*

1 1 0 0 0 1 0 1

4	2	1		1/2	1/4
1	0	1	.	0	1
2	1	0		-1	-2

$1 * 2^4 + 0 * 2^2 + 1 * 2^1 + 0 * 2^{-1} + 1 * 2^{-2}$

$(101.01)_2$

Binary Number System Conversion

Example: $(13)_{10}$

	Quotient	Remainder	Coefficient
$13 / 2 =$	6	1	$a_0 = 1$
$6 / 2 =$	3	0	$a_1 = 0$
$3 / 2 =$	1	1	$a_2 = 1$
$1 / 2 =$	0	1	$a_3 = 1$

Answer: $(13)_{10} = (a_3 a_2 a_1 a_0)_2 = (1101)_2$

↑
↑
 MSB LSB

Decimal (integer) to Binary

Example: $(0.625)_{10}$

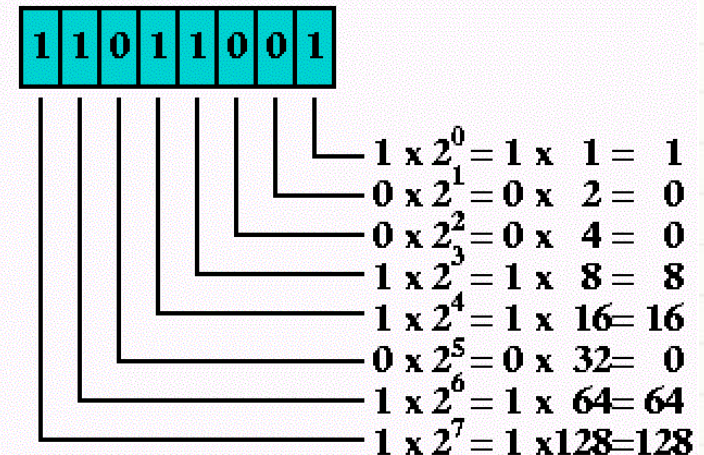
	Integer	Fraction	Coefficient
$0.625 * 2 =$	1	. 25	$a_{-1} = 1$
$0.25 * 2 =$	0	. 5	$a_{-2} = 0$
$0.5 * 2 =$	1	. 0	$a_{-3} = 1$

Answer: $(0.625)_{10} = (0.a_{-1} a_{-2} a_{-3})_2 = (0.101)_2$

↑
↑
 MSB LSB

Decimal (fraction) to Binary

Example: $(11011001)_2$



$$1 + 8 + 16 + 64 + 128 = 217$$

Binary to Decimal

Octal Number System

- Base = 8
 - 8 digits { 0, 1, 2, 3, 4, 5, 6, 7 }
- Weights
 - Weight = $(Base)^{Position}$
- Magnitude
 - Sum of "*Digit x Weight*"
- Formal Notation

64	8	1		1/8	1/64
5	1	2	•	7	4
2	1	0		-1	-2

$$\underline{5} * 8^2 + \underline{1} * 8^1 + \underline{2} * 8^0 + \underline{7} * 8^{-1} + \underline{4} * 8^{-2}$$
$$= (330.9375)_{10}$$
$$(\underline{512.74})_8$$

Octal Number System Conversion

Example: $(175)_{10}$ Decimal (integer) to Octal

	Quotient	Remainder	Coefficient
$175 / 8 =$	21	7	$a_0 = 7$
$21 / 8 =$	2	5	$a_1 = 5$
$2 / 8 =$	0	2	$a_2 = 2$

Answer: $(175)_{10} = (a_2 a_1 a_0)_8 = (257)_8$

Example: $(0.3125)_{10}$ Decimal (fraction) to Octal

	Integer	Fraction	Coefficient
$0.3125 * 8 =$	2	5	$a_{-1} = 2$
$0.5 * 8 =$	4	0	$a_{-2} = 4$

Answer: $(0.3125)_{10} = (0.a_{-1} a_{-2} a_{-3})_8 = (0.24)_8$

Hexadecimal Number System

- Base = 16
 - 16 digits { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F }
- Weights
 - Weight = (Base)^{Position}
- Magnitude
 - Sum of “Digit x Weight”
- Formal Notation

256	16	1		1/16	1/256
1	E	5	.	7	A
2	1	0		-1	-2

$$1 * 16^2 + 14 * 16^1 + 5 * 16^0 + 7 * 16^{-1} + 10 * 16^{-2}$$
$$= (485.4765625)_{10}$$
$$(1E5.7A)_{16}$$

Hexadecimal Number System Conversion

Example 1:

16	455	Remainders
16	28	7
16	1	12(C)
	0	1

Therefore, $(455)_{10} = (1C7)_{16}$

Decimal (integer) to Hex

Example 2:

$$0.565 \times 16 = 9.04 \quad 9$$

$$0.04 \times 16 = 0.64 \quad 0$$

$$0.64 \times 16 = 10.24 \quad 10 = A$$

$$0.24 \times 16 = 3.84 \quad 3$$

$$0.84 \times 16 = 13.44 \quad 13 = D$$

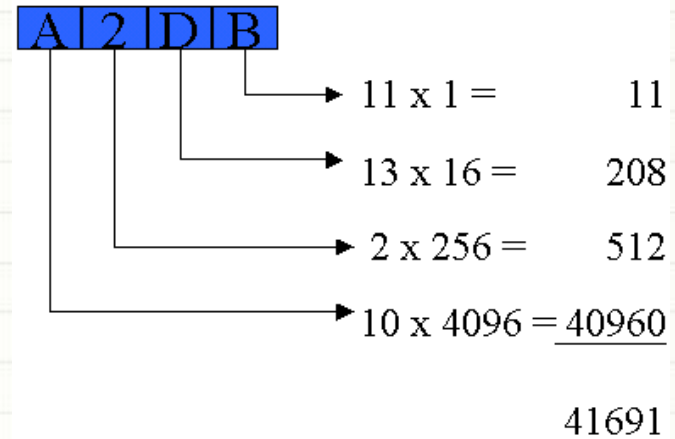
$$0.44 \times 16 = 7.04 \quad 7$$

$$0.04 \times 16 = 0.64 \quad 0$$

$$(0.565)_{10} = (0.90A3D70...)_{16}$$

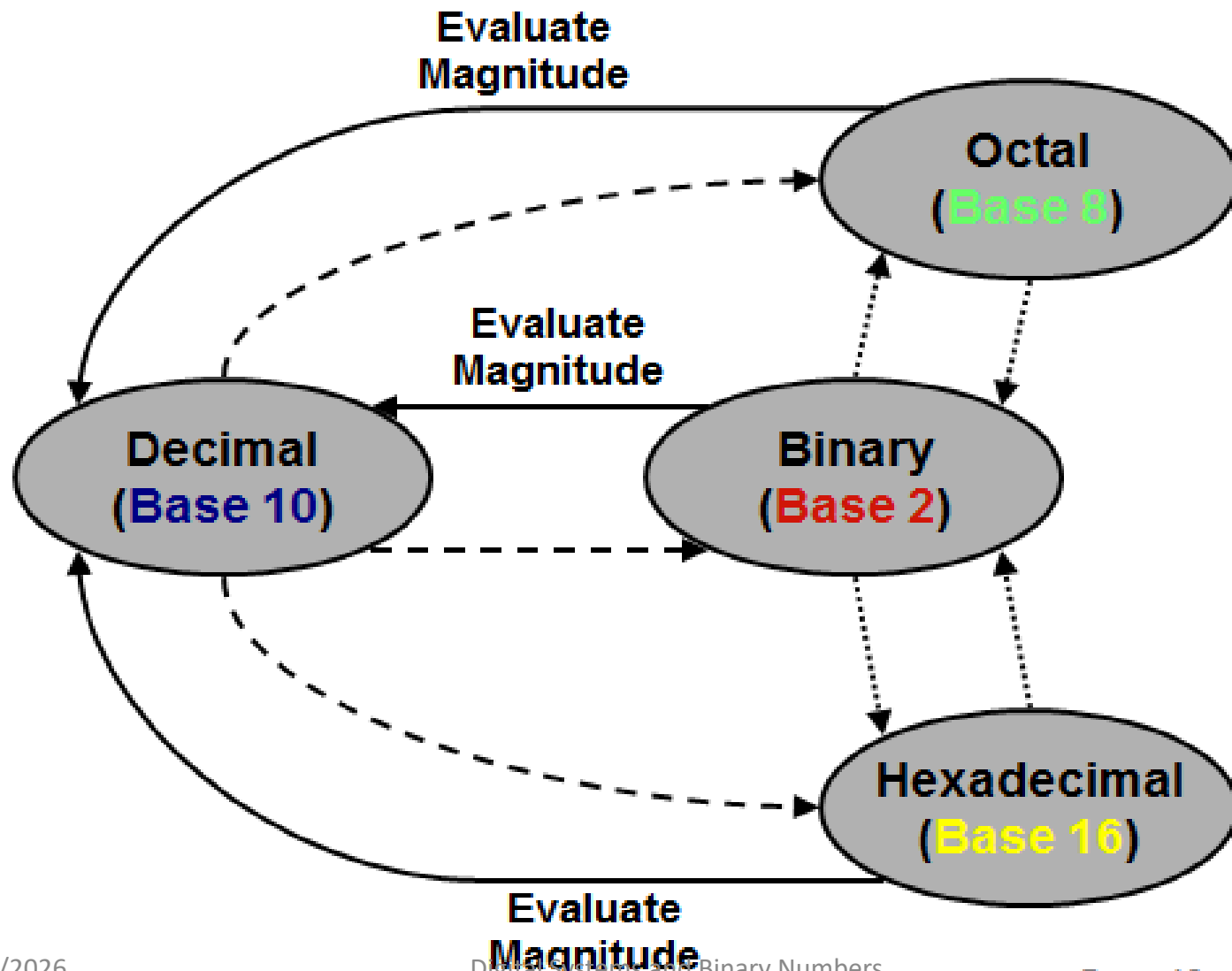
Decimal (fraction) to Hex

Example 3:



Hex to Decimal

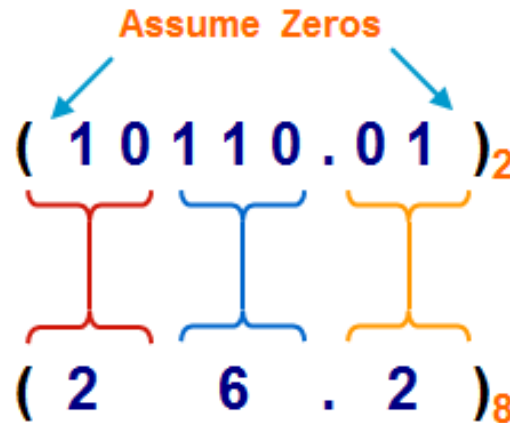
Number Base Conversions



Binary – Octal Conversion

- $8 = 2^3$
- Each group of 3 bits represents an octal digit.

Example:



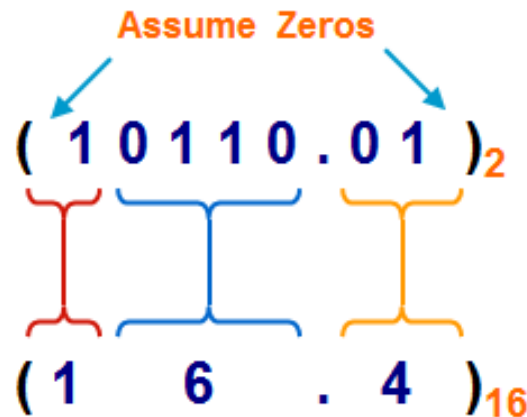
Works **both** ways (*Binary to Octal & Octal to Binary*)

Octal	Binary
0	0 0 0
1	0 0 1
2	0 1 0
3	0 1 1
4	1 0 0
5	1 0 1
6	1 1 0
7	1 1 1

Binary – Hexadecimal Conversion

- $16 = 2^4$
- Each group of 4 bits represents a hexadecimal digit.

Example:



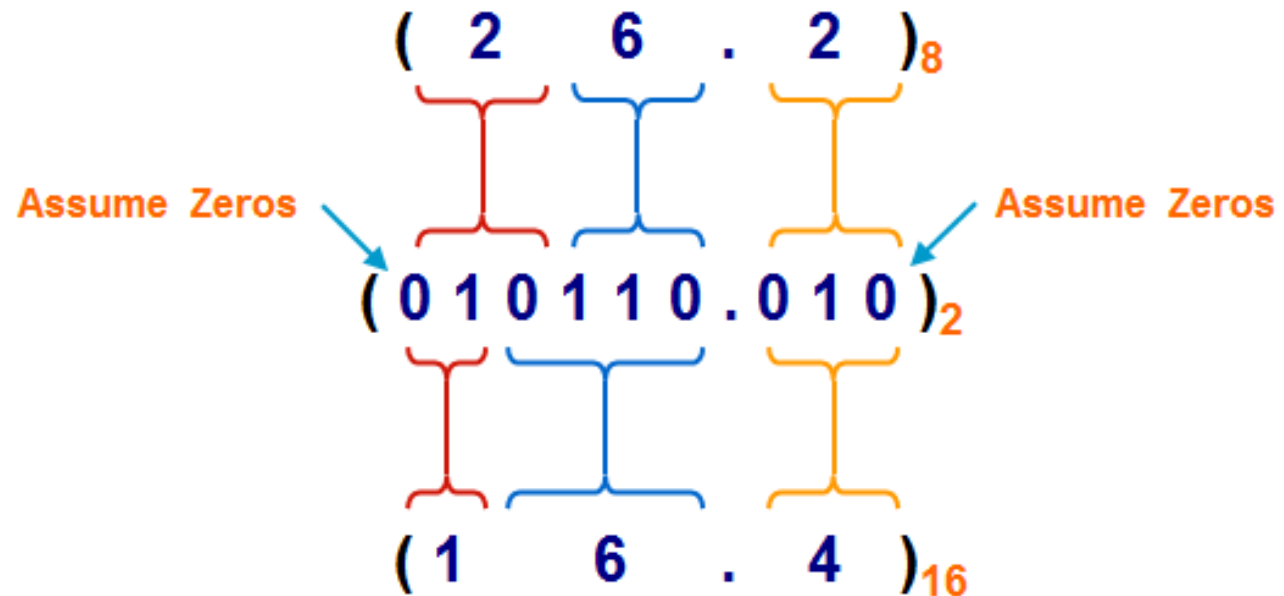
Works **both** ways (*Binary to Hex & Hex to Binary*)

Hex	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

Octal – Hexadecimal Conversion

- Convert to **Binary** as an intermediate step

Example:

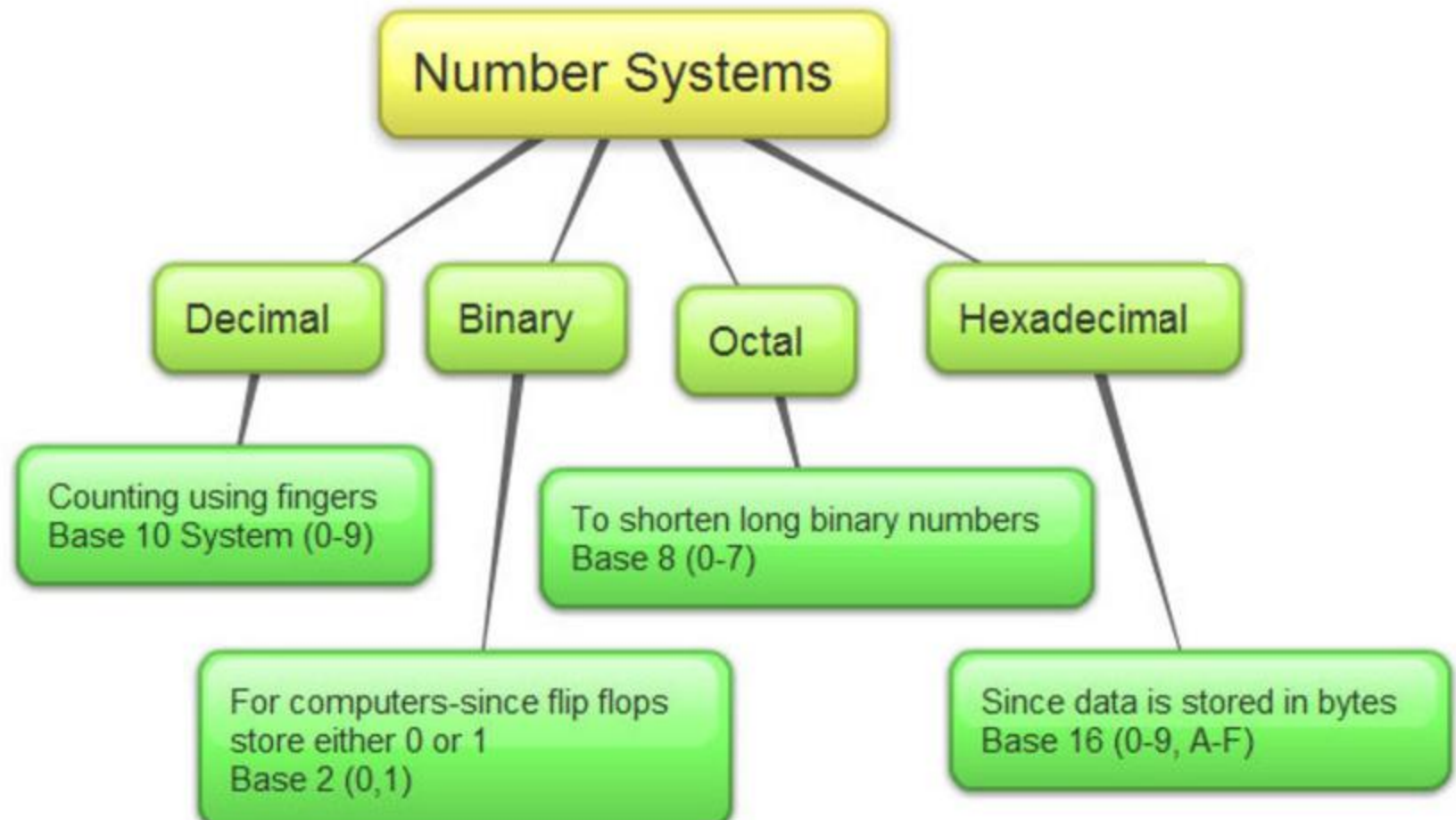


Works both ways (Octal to Hex & Hex to Octal)

Comparison

Decimal (base 10)	Binary (base 2)	Octal (base 8)	Hexadecimal (base 16)
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7
08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Applications



Binary Addition/Subtraction

- Example: Addition

$$\begin{array}{r}
 1\ 1\ 1\ 1\ 1\ 1 \\
 1\ 1\ 1\ 1\ 0\ 1 \\
 +\ 1\ 0\ 1\ 1\ 1 \\
 \hline
 1\ 0\ 1\ 0\ 1\ 0\ 0
 \end{array}
 \begin{array}{l}
 = 61 \\
 = 23 \\
 = 84
 \end{array}$$

$\geq (2)_{10}$

- Example: Subtraction

Borrow a “Base” when needed

$$\begin{array}{r}
 1\ 2\ 2\ 2 \\
 0\ 2\ 0\ 0\ 2 \\
 \cancel{1}\ 0\ 0\ \cancel{1}\ \cancel{1}\ 0\ 1 \\
 -\ 1\ 0\ 1\ 1\ 1 \\
 \hline
 0\ 1\ 1\ 0\ 1\ 1\ 0
 \end{array}
 \begin{array}{l}
 = (10)_2 \\
 = 77 \\
 = 23 \\
 = 54
 \end{array}$$

Octal Addition/Subtraction

- Example: Addition

$$456_8 + 123_8 = 601_8$$

1 1	carry
4 5 6	= 302 ₁₀
+ 1 2 3	= 83 ₁₀
6 0 1	= 385 ₁₀

- Example: Subtraction

$$456_8 - 173_8 = 333_8$$

8	borrow
³ 4 5 6	= 302 ₁₀
- 1 7 3	= 123 ₁₀
2 6 3	= 179 ₁₀

Hex Addition/Subtraction

- Example: Addition

$$\begin{array}{rcccc}
 & & & & \text{Carry} \\
 1 & 0 & 0 & 1 & \leftarrow \\
 & 8 & A & 5 & C \\
 & F & 3 & 9 & A \\
 \hline
 1 & 7 & D & F & 6
 \end{array}$$

- Example: Subtraction

When a borrow is required from the digit to the left, add 16 (decimal) to the current digit's value

$$\begin{array}{r}
 \boxed{16 + 5 = 21} \\
 \downarrow -1 \\
 \text{C675} \\
 - \text{A247} \\
 \hline
 \text{242E}
 \end{array}$$

Complements

- Complements are used in digital computers to **simplify the subtraction operation** and for logical manipulation. Simplifying operations leads to simpler, less expensive circuits to implement the operations.
- There are two types of complements for each base- r system:
 1. Radix complement.
 - » The r 's complement.
 - » Types are referred to as the 2's complement for binary numbers and the 10's complement for decimal numbers.
 2. Diminished radix complement.
 - » The $(r - 1)$'s complement.
 - » Types are referred to as the 1's complement for binary numbers and the 9's complement for decimal numbers.

Radix Complement

- The r 's complement
- **Example: 10's complement**
- The 10's complement of the first number is obtained by subtracting 8 from 10 in the least significant position and subtracting all other digits from 9

- The 10's complement of 012398 is 987602
- The 10's complement of 246700 is 753300

- **Example: 2's complement**
- The 2's complement of the first number is obtained by leaving the two least significant 0's and the first 1 unchanged and then replacing 1's with 0's and 0's with 1's

- The 2's complement of 1101100 is 0010100
- The 2's complement of 0110111 is 1001001

Diminished Radix Complement

- Diminished Radix Complement (r-1)'s Complement
- Example: 9's complement
 - It is obtained by subtracting each digit from 9.
 - 9's complement of 546700 is $999999 - 546700 = 453299$
- Example: 1's complement
 - All '0's become '1's and '1's become '0's
 - 1's complement of 1011000 is $1111111 - 1011000 = 0100111$

Complements

- Example:
 - Using 10's complement, subtract $72532 - 3250$.

$$\begin{array}{r} M = 72532 \\ \text{10's complement of } N = +96750 \\ \hline \text{Sum} = 169282 \\ \text{Discard end carry } 10^5 = -100000 \\ \hline \text{Answer} = 69282 \end{array}$$

- Example:
 - Using 10's complement, subtract $3250 - 72532$.

$$\begin{array}{r} M = 03250 \\ \text{10's complement of } N = +27468 \\ \hline \text{Sum} = 30718 \end{array}$$

There is no end carry.

Therefore, the answer is $-(10's \text{ complement of } 30718) = -69282$.

Binary Subtraction using 2's complement

- Convert number to be subtracted to its 2's complement form.
- Perform the addition.
- If the final carry is generated , then the result is positive and in true form.
- If the final carry is zero, result obtained is negative and in 2's complement form.

Complements

- Example:
 - Given the two binary numbers $X = 1010100$ and $Y = 1000011$, perform the subtraction (a) $X - Y$; and (b) $Y - X$, by using 2's complement.

$$\begin{array}{rcl} \text{(a)} & X = & 1010100 \\ & 2\text{'s complement of } Y = & \underline{+0111101} \\ & \text{Sum} = & 10010001 \\ & \text{Discard end carry } 2^7 = & \underline{-10000000} \\ & \text{Answer. } X - Y = & 0010001 \end{array}$$

$$\begin{array}{rcl} \text{(b)} & Y = & 1000011 \\ & 2\text{'s complement of } X = & \underline{+0101100} \\ & \text{Sum} = & 1101111 \end{array}$$

There is no end carry.
Therefore, the answer is
 $Y - X = - (2\text{'s complement of } 1101111) = -0010001.$

Binary Subtraction using 1's complement

- Convert number to be subtracted to its 1's complement form.
- Perform the addition.
- If the final carry is 1, then add it to the result obtained in step 2.
- If the final carry is zero, result obtained in step 2 is negative and in the 1's complement form.

Complements

- Subtraction of unsigned numbers can also be done by means of the $(r - 1)$'s complement. Remember that the $(r - 1)$'s complement is one less than the r 's complement.
- Example:**
 - Repeat Example 1.7, but this time using 1's complement.

(a) $X - Y = 1010100 - 1000011$

$$X = 1010100$$

$$1\text{'s complement of } Y = \pm 0111100$$

$$\text{Sum} = 10010000$$

$$\text{End-around carry} = \begin{array}{r} + \quad 1 \\ \hline \end{array}$$

$$\text{Answer. } X - Y = 0010001$$

(b) $Y - X = 1000011 - 1010100$

$$Y = 1000011$$

$$1\text{'s complement of } X = + 0101011$$

$$\text{Sum} = 1101110$$

There is no end carry,
Therefore, the answer is $Y - X$
 $= - (1\text{'s complement of } 1101110) = - 0010001.$

Signed and Unsigned Binary Numbers

- To represent negative integers, we need a notation for negative values.
- It is customary to represent the sign with a bit placed in the leftmost position of the number since binary digits.
- The convention is to make the **sign bit 0 for positive** and **1 for negative**.
- Example:

Signed-magnitude representation:	10001001
----------------------------------	----------

Signed-1's-complement representation:	11110110
---------------------------------------	----------

Signed-2's-complement representation:	11110111
---------------------------------------	----------

Signed and Unsigned Binary Numbers

Decimal	Signed-2's Complement	Signed-1's Complement	Signed Magnitude
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	—	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111

Binary Coded Decimal

- BCD Code
 - A number with k decimal digits will require 4k bits in BCD.
 - Decimal 396 is represented in BCD with 12bits as 0011 1001 0110, with each group of 4 bits representing one decimal digit.
 - A decimal number in BCD is the same as its equivalent binary number only when the number is between 0 and 9.
 - The binary combinations 1010 through 1111 are not used and have no meaning in BCD.

Table 1.4

Binary-Coded Decimal (BCD)

Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

BCD

- Example:
 - Consider decimal 185 and its corresponding value in BCD and binary:

➡ $(185)_{10} = (0001\ 1000\ 0101)_{\text{BCD}} = (10111001)_2$

- BCD addition

4	0100	4	0100	8	1000
<u>+5</u>	<u>+0101</u>	<u>+8</u>	<u>+1000</u>	<u>+9</u>	<u>+1001</u>
9	1001	12	1100	17	10001
			<u>+0110</u>		<u>+0110</u>
			10010		10111

BCD

- Example:
 - Consider the addition of $184 + 576 = 760$ in BCD:

BCD	1	1		
	0001	1000	0100	184
	<u>+ 0101</u>	<u>0111</u>	<u>0110</u>	+576
Binary sum	0111	10000	1010	
Add 6	<u> </u>	<u>0110</u>	<u>0110</u>	<u> </u>
BCD sum	0111	0110	0000	760

Other Decimal Codes

Four Different Binary Codes for the Decimal Digits

Decimal Digit	BCD 8421	2421	Excess-3	8, 4, -2, -1
0	0000	0000	0011	0000
1	0001	0001	0100	0111
2	0010	0010	0101	0110
3	0011	0011	0110	0101
4	0100	0100	0111	0100
5	0101	1011	1000	1011
6	0110	1100	1001	1010
7	0111	1101	1010	1001
8	1000	1110	1011	1000
9	1001	1111	1100	1111
Unused bit combi- nations	1010	0101	0000	0001
	1011	0110	0001	0010
	1100	0111	0010	0011
	1101	1000	1101	1100
	1110	1001	1110	1101
	1111	1010 compliments	1111	1110

ASCII

American Standard Code for Information Interchange.

- The parity bit is helpful in detecting errors during the transmission of information

ASCII A = 1000001

ASCII T = 1010100

With even parity

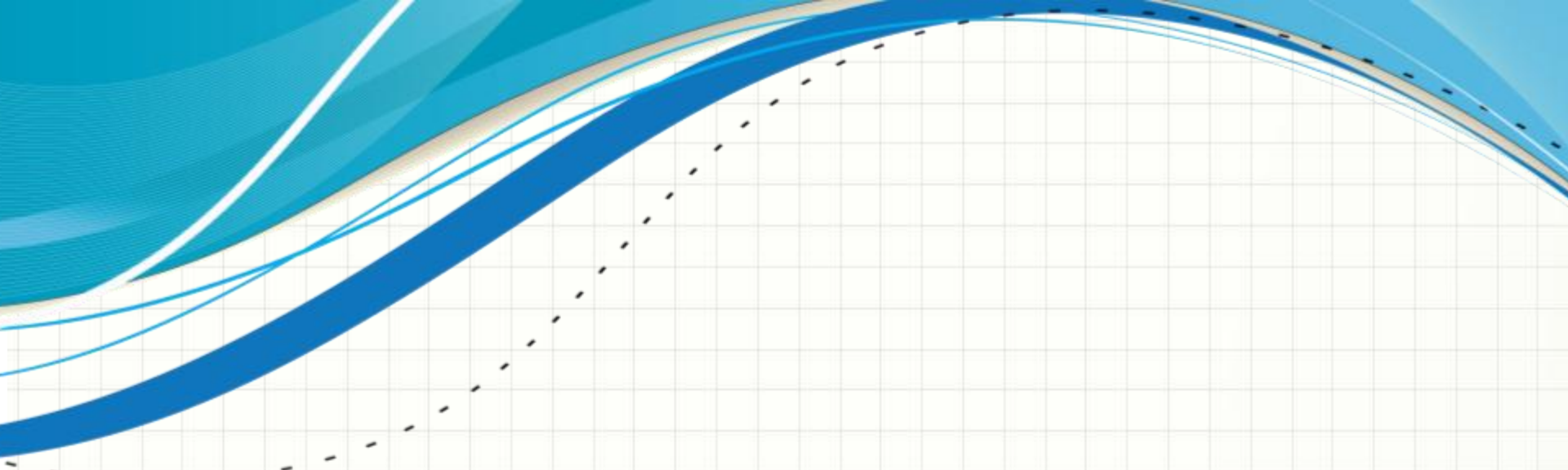
01000001

11010100

With odd parity

11000001

01010100



THANKS